

Reporte Técnico de Actividades Práctico- Experimentales Nro. 2

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Jose Camilo Merino Morocho
Asignatura	Matemáticas Discretas
Ciclo	1er Ciclo
Unidad	Unidad 1: Lógica Matemática
Resultado de aprendizaje de la unidad	Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001 FASE 2
Título de la Práctica	Proposiciones, Conectores Lógicos, Inferencia, Deducción y Tablas de Verdad
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	06 de abril al 15 de mayo de 2026
Horario	Lunes, martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	18 horas (3 horas por sesión APE)

2. Objetivo(s) de la Práctica

- Identificar y clasificar proposiciones lógicas simples y compuestas, distinguiendo entre proposiciones y enunciados no proposicionales.
- Comprender y aplicar los conectores lógicos fundamentales (negación, conjunción, disyunción, condicional, bicondicional) en la formulación de expresiones lógicas.
- Construir tablas de verdad para proposiciones compuestas, determinando su valor de verdad en todos los casos posibles.
- Aplicar las leyes de inferencia y deducción lógica para resolver problemas de razonamiento lógico en contextos computacionales y de ingeniería.
- Desarrollar habilidades de trabajo colaborativo, pensamiento crítico y resolución de problemas, integrando principios de equidad, inclusión y responsabilidad ética en el aprendizaje.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia
- Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad
- Guía de ejercicios de inferencia y deducción lógica
- Calculadora científica
- Material bibliográfico de apoyo (textos digitalizados de referencia)

4. Procedimiento / Metodología Ejecutada

FASE 2: Construcción de Tablas de Verdad y Operaciones Combinadas

1. El docente profundiza en la construcción de tablas de verdad para proposiciones compuestas con múltiples conectores, explicando el orden de evaluación (jerarquía de operadores) y el método sistemático para generar todas las combinaciones posibles de valores de verdad.
2. Los estudiantes construyen tablas de verdad para expresiones con 2, 3 y 4 variables proposicionales, identificando los casos en que la expresión resulta verdadera, falsa o es una tautología/contradicción/contingencia.
3. Práctica grupal: cada equipo resuelve ejercicios de simplificación de expresiones lógicas aplicando las leyes fundamentales (ley de identidad, complemento, idempotencia, conmutativa, asociativa, distributiva, de Morgan, absorción).
4. Los equipos analizan problemas de lógica computacional: cómo las expresiones booleanas se aplican en la evaluación de condiciones en programación (if-else, while, operadores lógicos en Python/C/Java).
5. Entrega del primer portafolio de ejercicios individuales (propuestas y tablas de verdad) a través de la plataforma EVA. Discusión sobre la ética en el uso de la lógica para la toma de decisiones algorítmicas y su impacto en la sociedad.

5. Resultados

1. TABALAS DE VERDAD

$$[(p \wedge q) \vee (p \wedge \neg q)] \wedge (p \vee r)$$

p	q	r	$\neg q$	$(p \wedge q)$	$(p \wedge \neg q)$	$(p \wedge q) \vee (p \wedge \neg q)$	$(p \vee r)$	Resultado Final
V	V	V	F	V	F	V	V	V
V	V	F	F	V	F	V	V	V
V	F	V	V	F	V	V	V	V
V	F	F	V	F	V	V	V	V
F	V	V	F	F	F	F	V	F



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

F	V	F	F	F	F	F	F	F
F	F	V	V	F	F	F	V	F
F	F	F	V	F	F	F	F	F

$$[\neg(p \vee q) \vee (\neg p \wedge q)] \rightarrow (\neg p \wedge q)$$

p	q	$\neg p$	$\neg q$	$p \vee q$	$\neg(p \vee q)$	$\neg p \wedge q$	$[\neg(p \vee q) \vee (\neg p \wedge q)]$	$(p \vee q)$
V	V	F	F	V	F	F	F	V
V	F	F	V	V	F	F	F	V
F	V	V	F	V	F	V	V	V
F	F	V	V	F	V	F	V	F

$$(p \vee q) \wedge (\neg r \vee s)$$

p	q	r	s	$p \vee q$	$\neg r$	$\neg r \vee s$	$(p \vee q) \wedge (\neg r \vee s)$
V	V	V	V	V	F	V	V
V	V	V	F	V	F	F	F
V	V	F	V	V	V	V	V
V	V	F	F	V	V	V	V
V	F	V	V	V	F	V	V
V	F	V	F	V	F	F	F
V	F	F	V	V	V	V	V
V	F	F	F	V	V	V	V
F	V	V	V	V	F	V	V
F	V	V	F	V	F	F	F
F	V	F	V	V	V	V	V
F	V	F	F	V	V	V	V
F	F	V	V	F	F	V	F
F	F	V	F	F	F	F	F
F	F	F	V	F	V	V	F
F	F	F	F	F	V	V	F

$$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)]$$

p	q	r	$(p \vee q \vee r)$	$(\neg p \wedge \neg q \wedge \neg r)$	$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)]$	$(q \wedge r)$	$(q \wedge \neg r)$	$(\neg q \wedge r)$	$[(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)]$	Resultado Final
V	V	V	V	F	V	V	F	F	V	V
V	V	F	V	F	V	F	V	F	V	V
V	F	V	V	F	V	F	F	V	V	V
V	F	F	V	F	V	F	F	F	F	F
F	V	V	V	F	V	V	F	F	V	V
F	V	F	V	F	V	F	V	F	V	V
F	F	V	V	F	V	F	F	V	V	V
F	F	F	F	V	V	F	F	F	F	F

2. EJERCICIOS DE LAS TABLAS DE VERDAD

Ejercicio 1

$$[(p \wedge q) \vee (p \wedge \neg q)] \wedge (p \vee r)$$

$$[p \wedge (q \vee \neg q)] \wedge (p \vee r) \rightarrow \text{Ley Distributiva}$$

$$[p \wedge 1] \wedge (p \vee r) \rightarrow \text{Ley Complemento}$$

$$p \wedge (p \vee r) \rightarrow \text{Ley de Absorción}$$

$$p$$

Ejercicio 2

$$[\neg(p \vee q) \vee (\neg p \wedge q)] \rightarrow (\neg p \wedge q)$$

$$[(\neg p \wedge \neg q) \vee (\neg p \wedge q)] \rightarrow (\neg p \wedge q) \rightarrow \text{Ley de Morgan}$$

$$[\neg p \wedge (\neg q \vee q)] \rightarrow (\neg p \wedge q) \rightarrow \text{Distributiva}$$

$$(\neg p \wedge V) \rightarrow (\neg p \wedge q) \rightarrow \text{De Complemento}$$

$$\neg p \rightarrow (\neg p \wedge q) \rightarrow \text{De Identidad}$$

$$\neg(\neg p) \vee (\neg p \wedge q) \rightarrow \text{De Implicación}$$

$$p \vee (\neg p \wedge q) \rightarrow \text{Doble negación}$$

$$(p \vee \neg p) \wedge (p \vee q) \rightarrow \text{Distributiva}$$

$$V \wedge (p \vee q) \rightarrow \text{De Complemento}$$

$$p \vee q$$

Ejercicio 3

$$\neg[(p \vee q) \wedge (q \vee r)] \vee (p \wedge 1) \vee (r \vee r)$$

$$(\neg p \wedge \neg q) \vee (\neg q \wedge \neg r) \vee (p \wedge 1) \vee (r \vee r) \rightarrow \text{Ley de Morgan}$$

$$(\neg p \wedge \neg q) \vee (\neg q \wedge \neg r) \vee p \vee (r \vee r) \rightarrow \text{Ley de Identidad}$$

$$(\neg p \wedge \neg q) \vee (\neg q \wedge \neg r) \vee p \vee r \rightarrow \text{Ley de Idempotencia}$$

$$p \vee (\neg p \wedge \neg q) \vee r \vee (\neg q \wedge \neg r) \rightarrow \text{Ley Asociativa}$$

$$(p \vee \neg p) \wedge (p \vee \neg q) \vee (r \vee \neg q) \wedge (r \vee \neg r) \rightarrow \text{Ley Distributiva}$$

$$1 \wedge (p \vee \neg q) \vee (r \vee \neg q) \wedge 1 \rightarrow \text{Ley de Complemento}$$

$$(p \vee \neg q) \vee (r \vee \neg q) \rightarrow \text{Ley de Identidad}$$

$$p \vee r \vee \neg q \vee \neg q \rightarrow \text{Ley Asociativa}$$

$$p \vee r \vee \neg q$$

Ejercicio 4

$$[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$$

$$[(p \vee q \vee r) \vee \neg(p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{L. de Morgan}$$

$$V \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley de Complemento}$$

$$V \wedge [q \wedge (r \vee \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley Distributiva}$$

$$V \wedge [q \wedge (V) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley de Complemento}$$

$$V \wedge [q \vee (\neg q \wedge r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley de Identidad}$$

$$V \wedge [(q \vee \neg q) \wedge (q \vee r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley Distributiva}$$

$$V \wedge [V \wedge (q \vee r)] \leftrightarrow (q \vee r) \rightarrow \text{Ley de Complemento}$$

$$V \wedge (q \vee r) \leftrightarrow (q \vee r) \rightarrow \text{Ley de Identidad}$$

$$(q \vee r) \leftrightarrow (q \vee r)$$

Ejercicio 5

$$[(p \wedge q) \vee (p \wedge \neg q)] \wedge [(p \wedge q) \vee (\neg p \wedge q)] \rightarrow \text{Ley distributiva}$$

$$[p \wedge (q \vee \neg q)] \wedge [q \wedge (p \vee \neg p)] \rightarrow \text{Ley de complemento}$$

$$(p \wedge F) \wedge (q \wedge F) \rightarrow \text{Ley de identidad}$$

$$p \wedge q$$

3. EXPRESIONES BOOLEANAS Y OPERADORES LOGICOS

En lenguajes como Python, C y Java, las expresiones booleanas se construyen utilizando operadores lógicos como AND, OR y NOT. Estos operadores permiten combinar múltiples condiciones para tomar decisiones más complejas. Por ejemplo:

- El operador AND (and) en Python, y en C/Java se utiliza cuando todas las condiciones deben cumplirse.
- El Operador OR (or / ||) se usa cuando basta con que una condición sea verdadera.
- Mientras que El Operador NOT (not / !) invierte el valor lógico de una expresión.

If-else

If sirve para tomar decisiones, si un programa pregunta algo y esta condición se cumple entonces ejecuta un bloque, si no, lo ignora. Else es el complemento de if y se ejecuta cuando la condición de if no se cumple.

```
1  #include <stdio.h>
2  int main(){
3
4      int edad;
5
6      printf ("Ingrese su edad: ");
7      scanf ("%i", &edad);
8
9      if (edad >= 18){
10     printf ("Eres mayor de edad");
11     } else {
12     printf ("Menor de edad");
13     }
14
15     return 0;
16 }
```

While

Sirve para repetir un bloque de código varias veces y se repite cuando la condición es verdadera, cuando esta no se cumple se detiene.

```
1  #include <stdio.h>
2  int main(){
3
4      int i = 1;
5
6      printf ("Contador del 1 al 5: \n");
7
8      while (i <= 5) {
9          printf ("%d\n", i);
10         i++;
11     }
12
13     return 0;
14
15 }
```

Ejercicios prácticos

Problema: Verifica si un número está entre 10 y 20.

Python	C / Java
<pre>num = 15 if num >= 10 and num <= 20: print("Está en el rango")</pre>	<pre>int num = 15; if (num >= 10 && num <= 20) { printf("Está en el rango"); }</pre>

Problema: Permitir acceso si el usuario es "admin" y la contraseña es correcta.

Python	Java
<pre>user = "admin" password = "1234" if user == "admin" and password == "1234": print("Acceso permitido")</pre>	<pre>String user = "admin"; String password = "1234"; if (user.equals("admin") && password.equals("1234")) { System.out.println("Acceso permitido"); }</pre>

6. Preguntas de Control

1. Explique la diferencia entre una proposición simple y una proposición compuesta. ¿Por qué es importante distinguir entre ambas en el contexto de la programación y diseño de algoritmos? Argumente con ejemplos concretos en lenguajes de programación

Proposición Simple: Es una afirmación única e indivisible que expresa una sola idea lógica. No contiene conectores. En programación, equivale a una sola comparación (por ejemplo, $x > 5$).

Proposición Compuesta: Es el resultado de unir dos o más proposiciones simples mediante conectores lógicos (AND, OR, NOT). Su valor de verdad depende de los valores de las proposiciones que la integran.

Distinguir entre ambas es vital por tres razones fundamentales:

Control de Flujo: Las estructuras de decisión (`if`) y repetición (`while`) dependen de estas evaluaciones para determinar qué camino sigue el programa.

Manejo de Complejidad: Los problemas reales rara vez dependen de una sola variable. Las proposiciones compuestas permiten validar múltiples requisitos simultáneamente (ej. "usuario autenticado" Y "tiene saldo").

Precisión Lógica: Un error en la construcción de una proposición compuesta (como confundir un `&&` con un `||`) puede causar que el software se comporte de forma errática o insegura.

Ejemplos Concretos

1. En Python (Legibilidad y Lógica)

- **Simple:** if temperatura > 30: (Solo evalúa un parámetro térmico).
- **Compuesta:** if temperatura > 30 and hay_humedad: (Aquí, el programa solo ejecutará la acción si **ambas** condiciones son verdaderas.)

2. En C / Java (Sintaxis Estricta)

- **Simple:** if (edad >= 18) (Evalúa un solo valor entero).
- **Compuesta:** if (edad >= 18 && tieneID == 1)

Usa el operador de cortocircuito &&. Si la primera parte (edad >= 18) es falsa, el programa ni siquiera evalúa la segunda, optimizando el rendimiento.

```
//Proposicion simple
if (edad >= 18 ){
    printf ("mayor de edad");
}

//Proposicion compleja
if (edad >= 18 && tieneLicencia) {
    printf ("puede Conducir");
}
```

6. Conclusiones

- Se concluye que, sin la distinción entre proposiciones simples y compuestas, sería imposible segmentar procesos complejos en decisiones binarias ejecutables por un procesador.
- La práctica demostró que una expresión lógica extensa es equivalente a una más simple (aplicando leyes como las de De Morgan). Esto permite que el procesador realice menos comparaciones, optimizando los recursos del sistema sin alterar el resultado final.
- La práctica demostró que el flujo de un algoritmo en lenguajes como Python, C o Java no es lineal, sino que depende estrictamente de la evaluación de valores de verdad

7. Recomendaciones

Incluya sugerencias para mejorar la práctica o su aplicación en casos reales.

Para aplicaciones en casos reales, se sugiere priorizar el orden de las proposiciones en condiciones compuestas. Colocar la condición con mayor probabilidad de ser falsa al principio de un AND (o la más probable de ser verdadera en un OR).

8. Reflexión crítica

Al escribir condiciones muy largas y redundantes; sin embargo, al aplicar técnicas de simplificación, aprendí que se puede lograr el mismo nivel de control con un código mucho más estético.

9. Anexos

① $[(p \wedge q) \vee (p \wedge q)] \wedge (p \vee r)$

$[p \wedge (q \vee q)] \wedge (p \vee r) \rightarrow$ Ley Distributiva

$[p \wedge 1] \wedge (p \vee r) \rightarrow$ Ley Complemento

$p \wedge (p \vee r) \rightarrow$ Ley de Absorción

$p //$

② $[\neg(p \vee q) \vee (\neg p \wedge q)] \rightarrow (\neg p \wedge q)$ Ley de Morgan

$[(\neg p \wedge \neg q) \vee (p \wedge q)] \rightarrow (\neg p \wedge q)$ Distributiva

$[\neg p \wedge (\neg q \wedge q)] \rightarrow (\neg p \wedge q)$ De Complemento

$(\neg p \wedge 1) \rightarrow (\neg p \wedge q)$ De Identidad

$\neg p \rightarrow (\neg p \wedge q)$ De Implicación

$\neg(\neg p) \vee (\neg p \wedge q)$ Doble negación

$p \vee (\neg p \wedge q)$ Distributivo

$(p \vee \neg p) \wedge (p \vee q)$ De Complemento

$1 \wedge (p \vee q)$ De Identidad

$p \vee q //$

③ $\neg[(p \vee q) \wedge (q \vee r)] \vee (p \wedge 1) \vee (r \vee r)$

$(\neg p \wedge \neg q) \vee (\neg q \wedge \neg r) \vee (p \wedge 1) \vee (r \vee r)$ Ley de Morgan

$(\neg p \wedge \neg q) \vee (\neg q \wedge \neg r) \vee p \vee (r \vee r)$ Ley de Identidad

$(\neg p \wedge \neg q) \vee (\neg q \wedge \neg r) \vee p \vee r$ Ley de Idempotencia

$p \vee (\neg q \wedge \neg r) \vee r \vee (\neg q \wedge \neg r)$ Ley asociativa

$(p \vee \neg q) \wedge (p \wedge \neg q) \vee (r \vee \neg q) \wedge (r \wedge \neg r)$ Ley distributiva

$1 \wedge (p \vee \neg q) \vee (r \vee \neg q) \wedge 1$ Ley de Complemento

$(p \vee \neg q) \vee (r \vee \neg q)$ Ley de Identidad

$p \vee r \vee \neg q \vee \neg q$ Ley Asociativa

$p \vee r \vee \neg q //$

④ $[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$
 $[(p \vee q \vee r) \vee \neg(p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley de Tercero
 $V \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley de Complemento
 $V \wedge [q \wedge (r \vee \neg r) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley distributiva
 $V \wedge [q \wedge (V) \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley de Complemento
 $V \wedge [q \vee (\neg q \wedge r)] \leftrightarrow (q \vee r)$ Ley de Identidad
 $V \wedge [(q \vee \neg q) \wedge (q \vee r)] \leftrightarrow (q \vee r)$ Ley distributiva
 $V \wedge [V \wedge (q \vee r)] \leftrightarrow (q \vee r)$ Ley de Complemento
 $V \wedge (q \vee r) \leftrightarrow (q \vee r)$ Ley De Identidad
 $(q \vee r) \leftrightarrow (q \vee r)$

⑤ $[(p \wedge q) \vee (p \wedge \neg q)] \wedge [(p \wedge q) \vee (\neg p \wedge q)]$ Ley Distributiva
 $[p \wedge (q \vee \neg q)] \wedge [q \wedge (p \vee \neg p)]$ Ley de Complemento
 $(p \wedge V) \wedge (q \wedge V)$ Ley de Identidad
 $p \wedge q$

① $[(p \wedge q) \vee (p \wedge \neg q)] \wedge (p \vee r)$

p	q	r	$\neg q$	$(p \wedge q)$	$(p \wedge \neg q)$	$(p \wedge q) \vee (p \wedge \neg q)$	$p \vee r$	R
V	V	V	F	V	F	V	V	V
V	V	F	F	V	F	V	V	V
V	F	V	V	F	V	V	V	V
V	F	F	V	F	V	V	V	V
F	V	V	F	F	F	F	F	F
F	V	F	F	F	F	F	F	F
F	F	V	V	F	F	F	F	F
F	F	F	V	F	F	F	F	F

es una contingencia

② $[\neg(p \vee q) \vee (\neg p \wedge q)] \rightarrow (\neg p \wedge q)$

p	q	$p \vee q$	$\neg(p \vee q)$	$\neg p$	$\neg p \wedge q$	$\neg(p \vee q) \vee (\neg p \wedge q)$	$(\neg p \wedge q)$
V	V	V	F	F	F	F	F
V	F	V	F	F	F	F	F
F	V	V	F	V	V	V	V
F	F	F	V	V	F	V	F

es una contingencia

③ $p \vee r \vee \neg q$

p	q	r	$p \vee r$	$\neg q$	$p \vee r \vee \neg q$
F	F	F	F	V	V
F	F	V	V	V	V
F	V	F	F	F	F
F	V	V	V	F	V
V	F	F	V	V	V
V	F	V	V	V	V
V	V	F	V	F	V
V	V	V	V	F	V

-----> es una contingencia

④ $[(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)] \wedge [(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)]$

p	q	r	$(p \vee q \vee r)$	$(\neg p \wedge \neg q \wedge \neg r)$	$(p \vee q \vee r) \vee (\neg p \wedge \neg q \wedge \neg r)$	$q \wedge r$	$q \wedge \neg r$	$\neg q \wedge r$	$[(q \wedge r) \vee (q \wedge \neg r) \vee (\neg q \wedge r)]$	R
V	V	V	V	F	V	V	F	F	V	V
V	V	F	V	F	V	F	V	F	V	V
V	F	V	V	F	V	F	F	V	V	V
V	F	F	V	F	V	F	F	F	F	F
F	V	V	V	F	V	V	F	F	V	V
F	V	F	V	F	V	F	V	F	V	V
F	F	V	V	F	V	F	F	V	V	V
F	F	F	V	V	V	F	F	F	F	F

Es una contingencia

⑤ $[(p \wedge q) \vee (p \wedge \neg q)] \wedge [(p \wedge q) \vee (\neg p \wedge q)]$

p	q	$\neg p$	$\neg q$	$p \wedge q$	$(p \wedge \neg q)$	$(\neg p \wedge q)$	$(p \wedge q) \vee (p \wedge \neg q)$	$(p \wedge q) \vee (\neg p \wedge q)$	R
V	V	F	F	V	F	F	V	V	V
V	F	F	V	F	V	F	V	F	F
F	V	V	F	F	F	V	F	V	F
F	F	V	V	F	F	F	F	F	F

Es una contingencia

Lógica Computacional: Como las expresiones booleanas se aplican en la evaluación de condiciones en programación.

En Lenguajes como Python, C y Java las expresiones booleanas se construyen utilizando operadores lógicos como AND, OR y NOT. Estos operadores permiten combinar múltiples condiciones para tomar decisiones más complejas. Por ejemplo:

El Operador AND (and) en Python y en C/Java se utiliza cuando todas las condiciones debe cumplirse

El Operador OR (or/||) se usa cuando basta con que una condición sea verdadera.

Mientras que El Operador NOT (not/!) invierte el valor lógico de una expresión

If-Else

If sirve para tomar decisiones si un programa pregunta algo y esta condición se cumple entonces ejecuta un bloque, si no lo logra. Else es el complemento de if y se ejecuta cuando la condición de if no se cumple.

```
1  #include <stdio.h>
2  int main () {
3
4      int edad;
5
6      printf ("Ingrese su edad ");
7      scanf ("%i", &edad);
8
9      if (edad >= 18) {
10         printf ("Eres mayor de edad");
11     } else {
12         printf ("Menor de edad")
13     }
14
15     return 0 ;
16 }
```

While

Sirve para repetir un bloque de código varias veces y se repite cuando la condición es verdadera, cuando esta no se cumple se detiene.

```
1 #include <stdio.h>
2 int main () {
3
4     int i = 1;
5
6     printf("Contador del 1 al 5")
7
8     while (i <= 5);
9         printf("%d\n", i);
10        i++;
11    }
12
13    return 0;
14
15 }
```

Ejercicios Practicos

Problema: Verifica si un numero esta entre 10 y 20

Python	C- PyJava
<pre>num = 15 if num >= 10 and num <= 20: printf("Esta en el rango")</pre>	<pre>int num = 15 if (num >= 10 && num <= 20) { printf("Esta en el rango"); }</pre>

Problema: Permitir acceso si el usuario es admin y la contraseña es correcta

Python	Java
<pre>User = "admin" Password = "123" if user == "admin" and password == "123": printf("Acceso permitido")</pre>	<pre>String user = "admin"; String password = "123"; if (user.equals("admin") && password.equals("1234")) { System.out.println("Acceso Permitido"); }</pre>